

Creating a LinkedList in Java

The **LinkedList** class in Java is part of the java.util package. It implements the **List** and **Deque** interfaces, making it a versatile choice for doubly linked list operations.

Syntax

```
LinkedList<Type> list = new LinkedList<>();
```

- Type is the type of elements (e.g., String, Integer, or any custom object).

Constructors

Default Constructor: Creates an empty LinkedList.

```
LinkedList<Type> list = new LinkedList<>();
```

Example:

```
import java.util.LinkedList;
public class Main
{
    public static void main(String[] args)
    {
        // Creating a LinkedList of Strings
        LinkedList<String> list = new LinkedList<>();
        list.add("Apple");
        list.add("Banana");
        System.out.println(list);    // Output: [Apple, Banana]
    }
}
```

From a Collection: Creates a LinkedList initialized with elements of a given collection.

```
LinkedList<Type> list = new LinkedList<>(Collection<? extends Type> c);
```

```
import java.util.LinkedList;
import java.util.ArrayList;
public class Main
{
    public static void main(String[] args)
    {
        // Creating an ArrayList
        ArrayList<String> arrayList = new ArrayList<>();
        arrayList.add("Dog");
        arrayList.add("Cat");

        // Creating a LinkedList from ArrayList
        LinkedList<String> list = new LinkedList<>(arrayList);
        System.out.println(list);    // Output: [Dog, Cat]
    }
}
```

Methods of LinkedList in Java

Add Elements

add(E element)

Adds an element at the end of the list.

Example:

```
LinkedList<String> list = new LinkedList<>();  
list.add("Apple");  
System.out.println(list);    // Output: [Apple]
```

add(int index, E element)

Adds an element at the specified index.

Example:

```
LinkedList<String> list = new LinkedList<>();  
list.add("Apple");  
list.add(1, "Banana");  
System.out.println(list);    // Output: [Apple, Banana]
```

addFirst(E element)

Adds an element at the beginning of the list.

Example:

```
LinkedList<String> list = new LinkedList<>();  
list.addFirst("Mango");  
System.out.println(list);    // Output: [Mango]
```

addLast(E element)

Adds an element at the end of the list.

Example:

```
LinkedList<String> list = new LinkedList<>();  
list.addLast("Grapes");  
System.out.println(list);    // Output: [Grapes]
```

```
import java.util.LinkedList;  
public class Main {  
    public static void main(String[] args)  
    {  
        LinkedList<String> list = new LinkedList<>();  
        list.add("Apple");  
        list.add(1, "Banana");  
        list.addFirst("Mango");  
        list.addLast("Grapes");  
        System.out.println(list);    // Output: [Mango, Apple, Banana, Grapes]  
    }  
}
```

Remove Elements

`remove()`

Removes the first element.

Example:

```
LinkedList<String> list = new LinkedList<>();  
list.add("Apple");  
list.add("Banana");  
list.remove();  
System.out.println(list);           // Output: [Banana]
```

`remove(int index)`

Removes the element at the specified index.

Example:

```
LinkedList<String> list = new LinkedList<>();  
list.add("Apple");  
list.add("Banana");  
list.remove(0);  
System.out.println(list);           // Output: [Banana]
```

Similarly,

`remove(Object o)` *Removes the first occurrence of the specified element.*

`removeFirst()` *Removes the first element.*

`removeLast()` *Removes the last element.*

```
import java.util.LinkedList;
```

```
public class Main
```

```
{  
    public static void main(String[] args)  
    {  
        LinkedList<String> list = new LinkedList<>();  
        list.add("Apple");  
        list.add("Banana");  
        list.remove("Apple");  
        list.add("Mango");  
        list.removeFirst();  
        System.out.println(list);     // Output: [Banana]  
    }  
}
```

Access Elements

get(int index)

Returns the element at the specified index.

Example:

```
LinkedList<String> list = new LinkedList<>();  
list.add("Apple");  
System.out.println(list.get(0));           // Output: Apple
```

getFirst()

Returns the first element.

Example:

```
LinkedList<String> list = new LinkedList<>();  
list.add("Apple");  
System.out.println(list.getFirst());       // Output: Apple
```

getLast()

Returns the last element.

Example:

```
LinkedList<String> list = new LinkedList<>();  
list.add("Apple");  
list.add("Banana");  
System.out.println(list.getLast());        // Output: Banana
```

Example Program:

```
import java.util.LinkedList;  
public class Main  
{  
    public static void main(String[] args)  
    {  
        LinkedList<String> list = new LinkedList<>();  
        list.add("Apple");  
        list.add("Banana");  
        System.out.println(list.get(1));    // Output: Banana  
        System.out.println(list.getFirst()); // Output: Apple  
        System.out.println(list.getLast()); // Output: Banana  
    }  
}
```

Other Common Methods

size()

Returns the number of elements in the list.

Example:

```
LinkedList<String> list = new LinkedList<>();  
list.add("Apple");  
System.out.println(list.size());           // Output: 1
```

contains(Object o)

Checks if the list contains a specified element.

Example:

```
LinkedList<String> list = new LinkedList<>();  
list.add("Apple");  
System.out.println(list.contains("Apple")); // Output: true
```

clear()

Removes all elements from the list.

Example:

```
LinkedList<String> list = new LinkedList<>();  
list.add("Apple");  
list.clear();  
System.out.println(list);           // Output: []
```

Iteration

Method	Description
iterator()	Returns an iterator over the elements.
listIterator()	Returns a list iterator over the elements.

```
import java.util.LinkedList;  
import java.util.Iterator;  
public class Main {  
    public static void main(String[] args)  
    {  
        LinkedList<String> list = new LinkedList<>();  
        list.add("Apple");  
        list.add("Banana");  
  
        Iterator<String> iterator = list.iterator();  
        while (iterator.hasNext()) {  
            System.out.println(iterator.next());  
        }  
    }  
}
```

```

    }
}
}

```

toArray()

Converts the list to an array.

```

import java.util.LinkedList;

public class Main
{
    public static void main(String[] args)
    {
        LinkedList<String> list = new LinkedList<>();
        list.add("Apple");
        list.add("Banana");

        Object[] array = list.toArray();
        for (Object obj : array) {
            System.out.println(obj); // Output: Apple, Banana
        }
    }
}

```

Differences Between ArrayList and LinkedList

Feature	ArrayList	LinkedList
Implementation	Backed by a dynamic array.	Backed by a doubly linked list.
Performance	- Faster for retrieving elements ($O(1)$ for get).	- Slower for retrieval ($O(n)$ for get).
Insertion/Deletion	- Slower ($O(n)$ in the worst case) due to shifting elements.	- Faster ($O(1)$ for addition/removal at the beginning or end).
Memory Usage	Requires less memory overhead since it stores elements in contiguous memory locations.	Requires more memory because it stores both data and pointers (next and previous node).
Use Case	Preferred when frequent access to elements is required.	Preferred when frequent insertion or deletion operations are performed.